

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CUỐI KỲ
MÔN XỬ LÝ NGÔN NGỮ TỰ NHIÊN NÂNG CAO

ĐỀ TÀI 05

“CHINESE–VIETNAMESE SENTENCE ALIGNMENT”

*Dóng hàng câu Hán – Việt cho Đại Việt Sử Ký Toàn Thư bằng CroCoAlign
Tải lập, đánh giá trên gold gán tay và cải tiến bằng giải mã đơn điệu*

NHÓM THỰC HIỆN: 25C11050 – Nguyễn Đình Lộc
25C15003 – Ngô Trương Minh Đạt
25C15015 – Trần Đắc Khoa

Dóng hàng câu Hán – Việt cho *Đại Việt Sử Ký Toàn Thư* bằng CroCoAlign: tái lập, đánh giá trên gold gán tay và cải tiến

Nguyễn Đình Lộc
25C11050

Ngô Trương Minh Đạt
25C15003

Trần Đắc Khoa
25C15015

Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Báo cáo cuối kỳ môn Xử lý ngôn ngữ tự nhiên nâng cao – Đề tài 05

Tóm tắt

Dóng hàng câu là bước đầu tiên để xây dựng ngữ liệu song song, và với các văn bản cổ thì đây thường là bước tốn công nhất. Trong báo cáo này chúng tôi thực hiện đề tài 05 trên cặp Hán cổ – Việt hiện đại của *Đại Việt Sử Ký Toàn Thư*: xây dựng ngữ liệu từ bản fulltext của nomfoundation.org (14 mục Ngoại kỷ, 1 703 câu Hán và 1 833 câu Việt), cài đặt và chạy lại hệ dóng hàng CroCoAlign [1] từ mã nguồn gốc với checkpoint chính thức, rồi đánh giá và tìm cách cải tiến. Thay vì chấm điểm bằng gold tự sinh từ chính mô hình, chúng tôi gán tay 185 nhóm liên kết trên ba mục làm tập kiểm tra, tách riêng tập phát triển, chọn cấu hình bằng cross-validation và so sánh với hai baseline không dùng mạng nơ-ron. Trên tập kiểm tra này CroCoAlign đạt F_1 strict 0,529 khi chạy nguyên bản và 0,565 sau tinh chỉnh. Khi giữ nguyên bộ mã hoá LaBSE của checkpoint nhưng thay bộ giải mã bằng quy hoạch động đơn điệu, kết quả tăng lên 0,918; một baseline chỉ dựa trên phiên âm Hán-Việt đạt 0,934. Kết quả cho thấy với những cặp văn bản dịch sát như ĐVSKTT, ràng buộc về thứ tự câu quan trọng hơn chất lượng biểu diễn ngữ nghĩa, đồng thời gold tự sinh từ một mô hình không thể dùng để đánh giá các mô hình cùng gốc với nó. Toàn bộ pipeline có thể chạy lại bằng một lệnh trên CPU.

1 Giới thiệu

Cho hai tài liệu là bản dịch của nhau, bài toán dóng hàng câu yêu cầu tìm các nhóm câu tương ứng giữa chúng, có thể là một-một, một-nhiều, nhiều-một, nhiều-nhiều, hoặc câu không có đối ứng. Đây là bước nền cho dịch máy, tra cứu song ngữ và số hoá văn bản cổ. Cặp Hán cổ – Việt của *Đại Việt Sử Ký Toàn Thư* (viết tắt ĐVSKTT) có vài đặc điểm khiến bài toán khó hơn bình thường. Văn bản Hán cổ không có dấu câu, nên ngay việc xác định thế nào là một “câu” đã là vấn đề. Khoảng cách ngôn ngữ và thời đại cũng lớn: các mô hình đa ngữ hiện nay được huấn luyện trên tiếng Trung hiện đại, không phải văn ngôn thế kỷ XV. Cuối cùng, bản dịch tiếng Việt chèn khá nhiều chú giải và niên đại, lại thường tách hoặc gộp câu, nên quan hệ giữa hai bản không thuần một-một.

Đề tài đặt ra ba yêu cầu: tiền xử lý dữ liệu cho phù hợp với mô hình, cài đặt và chạy lại CroCoAlign [1] theo bài báo EACL 2024, và thực nghiệm, đánh giá, nếu được thì cải tiến. Những đóng góp chính của chúng tôi gồm:

- một pipeline cào và tiền xử lý 14 mục ĐVSKTT, trong đó vấn đề tách câu Hán cổ được giải quyết nhờ dấu câu của phần phiên âm Hán-Việt đi kèm ở nguồn (Mục 4);
- việc tái lập CroCoAlign với checkpoint chính thức trên phiên bản torch 2.x và transformers 5.x mà không phải sửa mã gốc (Mục 5);
- một bộ gold gán tay gồm 185 nhóm liên kết, giao thức đánh giá tách DEV/TEST kết hợp cross-validation, một scorer độc lập tái hiện đúng `evaluate.py` gốc và hai baseline không dùng mạng nơ-ron (Mục 6);
- hai cải tiến giữ nguyên bộ mã hoá của CroCoAlign nhưng thay bộ giải mã bằng quy hoạch động đơn điệu, đưa F_1 strict từ 0,529 lên 0,918, cùng với bằng chứng định lượng cho thấy gold tự sinh từ mô hình bị thiên vị về cấu trúc (Mục 7 và 8).

2 Nghiên cứu liên quan

Các phương pháp đóng hàng câu sớm nhất dựa trên thông kê độ dài. Gale và Church [4] giả định tỉ lệ độ dài giữa câu nguồn và câu đích xấp xỉ phân phối chuẩn, rồi dùng quy hoạch động đơn điệu với các bước một-một, một-không, không-một, một-hai, hai-một và hai-hai để tìm phép đóng hàng có xác suất cao nhất. Dù đơn giản, phương pháp này vẫn là một baseline đáng kể khi hai bản đi song song với nhau. Các hệ sau đó như Hunalign hay Bleualign [5] bổ sung từ điển song ngữ hoặc bản dịch máy để có thêm tín hiệu về nội dung.

Với sự xuất hiện của các bộ nhúng câu đa ngữ, Vecalign [2] thay tín hiệu độ dài bằng độ tương đồng cosine giữa embedding LASER, dùng quy hoạch động xấp xỉ nhiều mức để giữ độ phức tạp tuyến tính, và định nghĩa bộ thước đo strict/lax mà CroCoAlign cũng như báo cáo này dùng lại. LaBSE [3] là bộ nhúng câu cho 109 ngôn ngữ được huấn luyện theo mục tiêu xếp hạng bản dịch, cho độ tương đồng giữa các cặp dịch cao và ổn định; đây là bộ mã hoá được CroCoAlign sử dụng.

CroCoAlign [1] là hệ đầu tiên giải quyết bài toán theo hướng hoàn toàn nơ-ron và có ngữ cảnh: mỗi cặp câu được một bộ phân loại quyết định dựa trên embedding đã được ngữ cảnh hoá theo tài liệu, không dùng quy hoạch động. Trên 20 cặp ngôn ngữ của Opus Books, hệ đạt macro- F_1 strict 87,5 so với 85,9 của Vecalign. Điểm khác của nghiên cứu này là dữ liệu nằm ngoài phân phối huấn luyện (Hán cổ, quan hệ gần như đơn điệu tuyệt đối) và việc kiểm chứng CroCoAlign bằng gold gán tay thay vì gold tự sinh.

3 Mô hình CroCoAlign

3.1 Kiến trúc

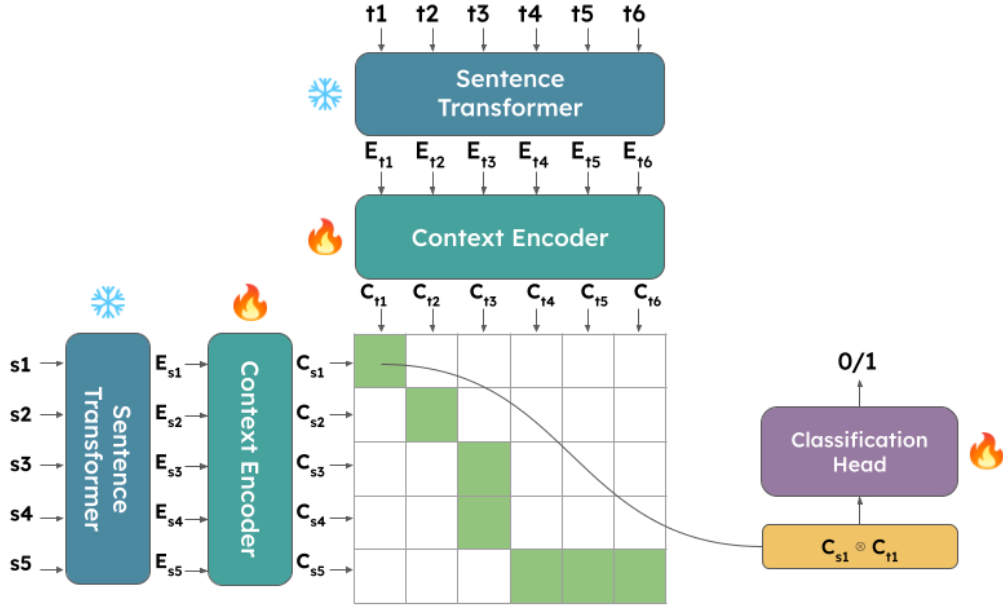
Hình 1 mô tả kiến trúc CroCoAlign theo bài báo; chúng tôi đã đối chiếu lại với mã nguồn trong `pl_modules/pl_module.py`. Cho dãy câu nguồn $s_1, \dots, s_n$ và câu đích $t_1, \dots, t_m$, mô hình gồm ba thành phần. Trước hết, bộ nhúng câu LaBSE (được giữ cố định trong suốt quá trình huấn luyện) sinh ra các vector $E_{s_i}, E_{t_j} \in \mathbb{R}^{768}$. Tiếp theo, một bộ mã hoá ngữ cảnh, thực chất là một Transformer theo cấu hình DistilBERT với 6 lớp, 6 đầu chú ý và dropout 0,2, được khởi tạo ngẫu nhiên và nhận vào chuỗi embedding câu của cả tài liệu (qua tham số `inputs_embeds`); nhờ positional embedding và cơ chế chú ý, đầu ra C_{s_i}, C_{t_j} mang thông tin về các câu lân cận. Đây là đóng góp chính của bài báo: mỗi câu được đặt trong ngữ cảnh trước khi so khớp. Cuối cùng, với mọi cặp (i, j) , tích Hadamard $C_{s_i} \odot C_{t_j}$ được đưa qua một đầu phân loại gồm Linear(768→384), dropout, ReLU và Linear(384→1), rồi qua sigmoid để ra xác suất M_{ij} ; cặp nào có $M_{ij} > 0,5$ được coi là liên kết. Cách này cho phép một câu nguồn nối với không, một hoặc nhiều câu đích và ngược lại.

3.2 Huấn luyện

Mô hình được huấn luyện với hàm mất mát binary cross-entropy trên toàn bộ ma trận, trong đó $\hat{M}$ là ma trận gold:

$$\mathcal{L} = -\frac{1}{nm} \sum_{i,j} \left[\hat{M}_{ij} \log \sigma(M_{ij}) + (1 - \hat{M}_{ij}) \log (1 - \sigma(M_{ij})) \right].$$

Dữ liệu huấn luyện lấy từ phần sách của dự án Opus, gồm 16 ngôn ngữ, 64 cặp ngôn ngữ và 251 cuốn sách, trong đó 231 cuốn dùng để huấn luyện, 10 để kiểm định và 10 để kiểm tra; khoảng 77% liên kết là một-một. Theo tệp cấu hình `conf/nn/default.yaml`, tác giả dùng AdamW với learning rate $5 \cdot 10^{-6}$ (bài báo ghi 10^{-5}), weight decay 0,01, warmup 10%, batch 64 câu, gradient clipping 10 và early stopping với patience 10 theo F_1 strict trên tập kiểm định; mô hình được huấn luyện trên một GPU RTX 3090 Ti. Checkpoint công bố nặng 2,31 GB, chứa cả LaBSE lẫn bộ mã



Hình 1: Kiến trúc CroCoAlign (hình lấy từ kho mã của tác giả [1]). Bộ nhúng câu được giữ cố định; bộ mã hoá ngữ cảnh và đầu phân loại được huấn luyện. Ma trận $n \times m$ với các ô xanh là những cặp được đóng hàng.

hoá ngữ cảnh và đầu phân loại. Chúng tôi không huấn luyện lại mà dùng nguyên checkpoint này, tức là chạy zero-shot trên cặp Hán cổ – Việt.

3.3 Suy luận trên tài liệu dài

Vì không thể đưa cả tài liệu vào bộ nhớ GPU, CroCoAlign suy luận theo từng lô 32 câu qua hai thủ tục. Thủ tục thứ nhất, gọi là *pointing*, dùng FAISS tìm k câu đích có cosine LaBSE cao nhất với câu đầu của lô nguồn, loại những ứng viên có vị trí tương đối trong tài liệu lệch quá ngưỡng $\text{min_dist} = 0,05$, rồi thử từng cửa sổ đích quanh các ứng viên còn lại và giữ cửa sổ cho nhiều liên kết nhất. Thủ tục thứ hai, *recovery*, xử lý trường hợp một câu nguồn bị nối với những câu đích không kề nhau bằng cách chọn lại theo cosine LaBSE, hoặc của riêng câu đó (labse), hoặc của câu kèm ba câu lân cận (labse-batch , biến thể tốt nhất trong bài báo). Cần lưu ý rằng trong cả quá trình này, mỗi cặp câu được quyết định độc lập với các cặp khác; không có ràng buộc toàn cục nào về thứ tự như trong quy hoạch động. Chi tiết này sẽ quan trọng khi phân tích kết quả.

4 Dữ liệu và tiền xử lý

4.1 Nguồn và cấu trúc

Bản fulltext ĐVSKTT của nomfoundation.org gồm 73 mục, mỗi mục là nhiều trang khắc in, được phân trang phía server bằng tham số $\text{POST}_{\text{curPg}}$. Mỗi trang có một bảng HTML: ô thứ nhất chứa chữ Hán và phiên âm Hán-Việt xen kẽ theo từng block, có dạng

<chữ Hán> [trang*dòng*cột] <phiên âm Hán-Việt, có dấu câu>

còn ô thứ hai (“Dịch Quốc Ngữ”) là bản dịch tiếng Việt hiện đại theo cùng thứ tự. Một block điển hình trông như sau:

按黃帝時建萬國以交趾界於西南遠在百粵之表 [1a*4*1] *Án: Hoàng đế thời, kiến vạn quốc, dĩ Giao Chỉ giới ư Tây Nam, viễn tại Bách Việt chi biểu.*
→ “Xét: Thời Hoàng Đế dựng muôn nước, lấy địa giới Giao Chỉ về phía Tây Nam, xa ngoài đất Bách Việt.”

4.2 Hai quan sát quyết định cách làm

Quan sát thứ nhất liên quan đến việc tách câu. Người biên soạn bản phiên âm đã đặt dấu chấm, dấu hỏi ở cuối mỗi đoạn phiên âm, và mỗi block chữ Hán ứng đúng với một đoạn như vậy. Nói cách khác, ranh giới câu của văn bản Hán cổ đã được đánh dấu gián tiếp qua phần phiên âm, nên chúng tôi có thể coi mỗi block là một câu mà không cần đến mô hình tách câu riêng cho văn ngôn.

Quan sát thứ hai là phiên âm Hán-Việt còn đóng vai trò cầu nối từ vựng giữa hai bản. Bản dịch hiện đại giữ nguyên rất nhiều từ Hán-Việt, đặc biệt là tên người, địa danh, chức quan và niên hiệu (*Sĩ Nhiếp, Giao Châu, Thái thú, Kiến An, Long Biên*), nên độ trùng lặp âm tiết giữa phiên âm và câu dịch là một tín hiệu đo được mà không cần mô hình nào. Chúng tôi dùng tín hiệu này cho cả việc gán nhãn tay (Mục 6.1) lẫn baseline (Mục 6.4).

4.3 Pipeline



Hình 2: Pipeline tiền xử lý. Hai tệp `zh.jsonl` và `vi.jsonl` theo đúng định dạng đầu vào của CroCoAlign (mỗi dòng `{"ids": [...], "text": [...]}`); `blocks.tsv` giữ lại phiên âm để phục vụ gán nhãn và baseline.

Script `scrape_dvsktt.py` có thể chạy với `requests` hoặc chỉ với `urllib` của thư viện chuẩn (tự dựng POST multipart), có cơ chế thử lại và nghỉ 0,8 giây giữa các trang. Script `preprocess.py` tách block bằng biểu thức chính quy trên marker `[\d+[ab]*\d+\d+]`, giữ lại ký tự CJK (kể cả các vùng mở rộng B và F), bỏ nội dung trong ngoặc vuông của bản dịch (chú giải, niên đại) rồi tách câu Việt theo các dấu `. ! ? ;`. Bảng 1 thống kê 14 mục đã xử lý.

Bảng 1: 14 mục Ngoại kỷ sau tiền xử lý. DEV gồm mục 1–3, TEST gồm mục 7, 9, 10.

Mục	Vai trò	Trang	Hán	Việt	Mục	Vai trò	Trang	Hán	Việt
1 Hồng Bàng thị	DEV	9	80	90	8 thuộc Ngô–Tề	–	27	309	312
2 Nhà Thục	DEV	13	112	125	9 Tiền Lý	TEST	6	64	65
3 Nhà Triệu	DEV	35	331	354	10 Triệu Việt Vương	TEST	6	51	59
4 thuộc Tây Hán	–	2	22	24	11 Hậu Lý	–	6	44	46
5 Trưng Nữ Vương	–	2	19	22	12 thuộc Tuỳ–Đường	–	33	350	357
6 thuộc Đông Hán	–	9	83	94	13 Nam Bắc phân tranh	–	5	56	55
7 Sĩ Vương	TEST	11	75	91	14 Nhà Ngô	–	14	107	139

Tổng: 190 trang, **1 703** câu Hán, **1 833** câu Việt

Dữ liệu thực tế có một số nhiễu cần lưu ý. Bản dịch có nhiều câu hơn bản Hán khoảng 8% do chú giải và tách câu. Một vài câu bị cắt ngay ở nguồn (*“KỶ TRIỆU V”*, *“là Ki”*). Có những câu bị cắt ngang ở ranh giới trang trong cả hai bản (*“Thời Thứ sử Chu”* | *“Phù bị giặc Di giết...”*). Cũng có những chú giải rất dài, chẳng hạn truyền thuyết Chữ Đồng Tử được dịch thành sáu câu Việt cho một block Hán.

5 Cài đặt và chạy lại CroCoAlign

Mã nguồn CroCoAlign được đưa vào kho của chúng tôi dưới dạng git submodule, ghim ở commit `2e93992` (HEAD của upstream, tháng 9/2024) để bảo đảm tái lập. Tệp `requirements.txt` của repo ghim `torch==1.11`, `pytorch-lightning==1.5`, `transformers==4.20` và `faiss-gpu==1.7.2`, tất cả đều không cài được trên phần cứng hiện có (GPU kiến trúc

Blackwell sm_120 và máy macOS arm64). Chúng tôi vì vậy phải dùng các phiên bản mới hơn nhiều, kéo theo một loạt điểm không tương thích được liệt kê trong Bảng 2. Nguyên tắc xuyên suốt là không sửa mã gốc: mọi vá lỗi đều nằm trong các wrapper ở thư mục `scripts/` (`run_align.py`, `run_eval.py`, `run_eval_tuned.py`).

Bảng 2: Các vướng mắc khi cài đặt và cách khắc phục

Vấn đề	Nguyên nhân	Khắc phục
torch ghim không chạy	RTX 5050 (sm_120) không tương thích <code>torch==1.11</code> ; macOS không có CUDA	torch 2.11+cu128 (WSL2) / torch 2.14 CPU (macOS)
faiss-gpu không cài được	chỉ có bản cho CUDA cũ	faiss-cpu; dữ liệu vài nghìn câu, đủ nhanh
Nạp checkpoint lỗi	PL 2.6 bỏ <code>_load_model_state</code> mà <code>nn_core.load_model</code> gọi; torch 2.6 mặc định <code>weights_only=True</code> ; <code>sentence-transformers</code> ≥ 3 đổi tên module <code>auto_model</code> $\rightarrow$ <code>model</code>	loader tự viết: giải nén <code>.ckpt.zip</code> , <code>torch.load(weights_only=False)</code> , ánh xạ lại khoá <code>state_dict</code> ; kết quả <code>missing=0</code> , chỉ dư 2 <code>buffer position_ids</code> không ảnh hưởng
Thiếu phụ thuộc <code>nn-core</code>	phải cài <code>--no-deps</code> để không bị ép <code>lightning</code> cũ	thêm <code>GitPython</code> , <code>python-dotenv</code>
Đường dẫn tương đối	<code>evaluate.py</code> ghi <code>results_*.tsv</code> vào thư mục hiện hành	runner chuyển vào thư mục đích và dùng đường dẫn tuyệt đối

Để kiểm chứng, chúng tôi chạy trước ví dụ EN–IT kèm theo repo và thu được kết quả đúng, kể cả với câu tiêu đề không có đối ứng. Sau đó pipeline Hán–Việt được chạy trọn vẹn với checkpoint chính thức trên hai môi trường: WSL2 với GPU RTX 5050 (`env crocoalign`: Python 3.10, torch 2.11+cu128, PL 2.6.5) và macOS arm64 chỉ dùng CPU (`.venv`: Python 3.10, torch 2.14, `transformers` 5.16, `sentence-transformers` 6.0, PL 2.6.5, `faiss-cpu`, `nn-template-core` 0.1.1). Số liệu trong báo cáo lấy từ lần chạy trên macOS. Toàn bộ phần cần torch, gồm CroCoAlign nguyên bản và tinh chỉnh trên tập TEST cùng các cải tiến trên 14 mục với 64 cấu hình, chạy hết trong khoảng 15 phút CPU; riêng mỗi lượt CroCoAlign trên 190 câu TEST mất khoảng 5 phút kể cả thời gian nạp mô hình.

6 Phương pháp đánh giá và cải tiến

6.1 Gold gán tay

Ở phiên bản đầu của đề tài, chúng tôi chấm CroCoAlign bằng một bộ gold “bạc” sinh tự động từ cosine LaBSE và quy hoạch động đơn điệu, đồng thời chọn siêu tham số trên chính tập dùng để báo cáo. Nhìn lại, cách làm này có hai lỗi: đánh giá bị vòng tròn vì CroCoAlign cũng dựa trên LaBSE, và siêu tham số bị tinh chỉnh trên tập kiểm tra. Thiết kế cuối cùng (Bảng 3) sửa cả hai điểm.

Ba mục TEST (7, 9 và 10, tổng cộng 190 câu Hán và 215 câu Việt) được gán nhãn thủ công bằng cách đọc phiên âm Hán–Việt đối chiếu với bản dịch. Chúng tôi theo năm quy tắc: liên kết dựa trên nội dung chứ không dựa trên vị trí; nhóm phải nhỏ nhất có thể; câu bị cắt ngang ở cùng một chỗ trong hai bản thì ghép từng mảnh, cắt lệch thì gộp thành nhóm nhiều-nhiều; chú giải, niên đại và tiêu đề thuộc về block chứa chúng, nếu bản dịch không có thì để trống; và mỗi câu chỉ xuất hiện đúng một lần (điều kiện này được `annot2gold.py` kiểm tra tự động). Kết quả là 185 nhóm, gồm 160 nhóm một-một, 10 nhóm một-hai, 5 nhóm một-ba, 2 nhóm một-sáu hoặc một-bảy, 3 nhóm hai-một, 2 nhóm hai-hai và 3 câu Hán không có đối ứng. Mọi quyết định không phải một-một đều được chú

thích lý do trong `data/annotation/*.align.txt`; tiêu chí và quy trình rà soát độc lập được ghi trong `data/annotation/README.md`. Bộ gold này không phụ thuộc vào LaBSE hay CroCoAlign dưới bất kỳ hình thức nào.

Bảng 3: Thiết kế DEV / TEST

	DEV	TEST
Mục Gold	1 Hồng Bàng, 2 Nhà Thục, 3 Nhà Triệu silver: DP đơn điệu trên cosine LaBSE (<code>build_silver.py</code>)	7 Sĩ Vương, 9 Tiền Lý, 10 Triệu Việt Vương gán tay, 185 nhóm
Quy mô	523 / 569 câu	190 / 215 câu
Dùng để	chọn <code>min_dist</code> và ngưỡng của Cro-CoAlign tinh chỉnh	báo cáo; cấu hình các hệ DP chọn bằng CV <code>leave-one-section-out</code>

6.2 Thước đo

Chúng tôi dùng bộ thước đo của Vecalign [2], cũng là thước đo trong `evaluate.py` của CroCoAlign. Một liên kết dự đoán được tính là đúng theo nghĩa strict nếu trùng hoàn toàn với một nhóm gold, cả về tập câu nguồn lẫn tập câu đích; theo nghĩa lax, chỉ cần có ít nhất một câu nguồn và một câu đích chung với nhóm gold chứa câu nguồn đó. Precision tính trên các liên kết dự đoán (kể cả liên kết rỗng), recall tính trên các nhóm gold không rỗng, F_1 là trung bình điều hoà, và điểm cuối là trung bình của ba mục. Script `scripts/score.py` tái hiện đúng logic này bằng Python thuần; chúng tôi đã đối chiếu với `evaluate.py` trên sáu tổ hợp (ba mục, hai cấu hình) và các con số khớp đến ba chữ số thập phân. Nhờ vậy mọi hệ đều được chấm trên cùng một thước đo, và việc chấm không cần GPU.

6.3 Chọn cấu hình bằng cross-validation

Với CroCoAlign, hai siêu tham số suy luận là `min_dist` và ngưỡng quyết định được chọn trên DEV. Với các hệ dùng quy hoạch động thì DEV không dùng được cho mục đích này, vì bộ gold bạc của DEV được sinh bởi đúng thuật toán “cosine LaBSE cộng quy hoạch động”: hệ LaBSE+DP đạt $F_1 = 1,000$ trên DEV với mọi cấu hình gần cấu hình gốc, và DEV vì thế luôn chọn trọng số $w = 1$. Chúng tôi thay bằng cross-validation theo kiểu bỏ-một-mục trên chính gold gán tay: với mỗi mục TEST, cấu hình được chọn là cấu hình có F_1 strict trung bình cao nhất trên hai mục còn lại, sau đó mới chấm mục bị giữ lại. Không mục nào tự chọn cấu hình cho mình (`pick_config.py --cv`). Kết quả khi chọn trên DEV vẫn được giữ trong bảng để đối chiếu.

6.4 Hai baseline không dùng mạng nơ-ron

Cả hai baseline dùng chung bộ giải mã quy hoạch động ở Mục 6.5 và chỉ khác nhau ở ma trận tương đồng S . Baseline độ dài theo Gale–Church đặt $S_{ij} = \exp(-z_{ij}^2/2)$ với $z_{ij} = (\log(\ell_j^v/\ell_i^h) - \log \bar{r})/\sigma$, trong đó ℓ^h là số chữ Hán, ℓ^v là số âm tiết Việt, $\bar{r}$ là tỉ lệ độ dài trung bình của cả mục và $\sigma = 0,6$. Baseline từ vựng Hán-Việt dùng hệ số Dice có trọng số IDF giữa tập âm tiết A_i của phiên âm và tập âm tiết B_j của câu Việt (sau khi chuyển về chữ thường, bỏ dấu câu và chữ số):

$$S_{ij} = \frac{2 \sum_{w \in A_i \cap B_j} \text{idf}(w)}{\sum_{w \in A_i} \text{idf}(w) + \sum_{w \in B_j} \text{idf}(w)}, \quad \text{idf}(w) = \log \frac{N + 1}{\text{df}(w) + 0,5},$$

với N là tổng số câu của mục. Trọng số IDF làm giảm ảnh hưởng của hư từ (*chi, nhi, đã* ở phía Hán; *và, của* ở phía Việt) và nhấn mạnh tên riêng.

6.5 Bộ giải mã đơn điệu và hai cải tiến

Gọi $D(i, j)$ là điểm đúng hàng tốt nhất giữa i câu Hán đầu và j câu Việt đầu. Ta có công thức truy hồi

$$D(i, j) = \max \left\{ \begin{array}{l} D(i-1, j-1) + S_{ij} - \tau; \quad D(i-1, j) - \gamma; \quad D(i, j-1) - \gamma; \\ D(i-1, j-2) + S_{i, \{j-1, j\}} - \tau - \mu; \quad D(i-2, j-1) + S_{\{i-1, i\}, j} - \tau - \mu \end{array} \right\},$$

trong đó τ là ngưỡng (một cặp có tương đồng thấp hơn τ sẽ cho điểm âm, tức là thà để trống còn hơn), γ là phạt cho câu không có đối ứng và $\mu = \gamma$ là phạt gộp. Điểm gộp $S_{i, \{j-1, j\}}$ có thể tính theo hai cách: lấy trung bình hai ô ($c = 0$), hoặc so sánh câu Hán với hai câu Việt đã ghép lại thành một ($c = 1$). Với baseline từ vựng, cách thứ hai là Dice trên hợp của hai tập âm tiết; với LaBSE, đó là cosine với embedding của chuỗi ghép, nên cần mã hoá thêm $m - 1$ cặp câu Việt kề nhau và $n - 1$ cặp câu Hán kề nhau. Truy vết cho ra các nhóm một-một, một-không, không-một, một-hai và hai-một. Độ phức tạp là $O(nm)$; mục dài nhất (350×357) chạy dưới một giây.

Cải tiến thứ nhất, mà chúng tôi gọi là LaBSE(ckpt)+DP, lấy chính bộ LaBSE trong checkpoint CroCoAlign (`model.transformer`) để mã hoá mọi câu, đặt $S = \cos$ và giải mã bằng quy hoạch động ở trên. Nói cách khác, mọi thứ của CroCoAlign được giữ nguyên, chỉ có bộ giải mã ở Mục 3.3 bị thay. Cải tiến thứ hai kết hợp thêm tín hiệu từ vựng, $S = w \cdot \cos_{\text{LaBSE}} + (1 - w) \cdot S^{\text{lex}}$, với cùng bộ giải mã. Lưới cấu hình của baseline gồm $\tau \in \{0,10; 0,15; 0,20\}$, $\gamma \in \{0,05; 0,10\}$ và $c \in \{0, 1\}$ (12 cấu hình); của các hệ LaBSE gồm thêm $w \in \{1; 0,7; 0,5; 0,3\}$ và $\tau \in \{0,15; 0,25; 0,35; 0,45\}$ (64 cấu hình, trong đó $w = 1$ chính là cải tiến thứ nhất). Embedding được lưu lại trong `data/emb/*.npz` (4 MB) nên toàn bộ lưới có thể tái sinh mà không cần checkpoint.

7 Kết quả

Bảng 4: Kết quả trên TEST (gold gán tay, 185 nhóm), trung bình ba mục; s là strict, l là lax. Cấu hình của các hệ DP được chọn bằng CV (Mục 6.3); CV chọn $c = 1$ ở mọi fold của mọi hệ.

Hệ thống	P_s	R_s	$F_{1,s}$	P_l	R_l	$F_{1,l}$
CroCoAlign nguyên bản (<code>min_dist 0,05</code> , ngưỡng 0,5)	0,536	0,522	0,529	0,648	0,642	0,645
CroCoAlign tinh chỉnh (0,15 / 0,20, chọn trên DEV)	0,569	0,561	0,565	0,669	0,669	0,669
Gale–Church độ dài + DP	0,869	0,890	0,879	0,947	0,965	0,956
Hán-Việt lexical + DP	0,930	0,939	0,934	0,993	1,000	0,997
Cải tiến 1: LaBSE(ckpt) + DP	0,914	0,922	0,918	0,995	0,995	0,995
Cải tiến 2: LaBSE + Hán-Việt ($w = 0,3$) + DP	0,923	0,932	0,927	0,993	0,993	0,993
<i>Đối chứng:</i> cải tiến 1, cấu hình chọn trên DEV silver	0,834	0,854	0,844	0,957	0,972	0,965

Bảng 5: F_1 strict / lax theo từng mục TEST, và trên DEV silver để tham khảo

Hệ thống	7 Sĩ Vương	9 Tiền Lý	10 Triệu Việt Vương	DEV silver
CroCoAlign nguyên bản	0,489 / 0,629	0,587 / 0,619	0,510 / 0,688	0,609 / 0,669
CroCoAlign tinh chỉnh	0,565 / 0,648	0,619 / 0,651	0,510 / 0,708	0,646 / 0,717
Gale–Church độ dài + DP	0,801 / 0,938	1,000 / 1,000	0,837 / 0,929	0,849 / 0,941
Hán-Việt lexical + DP	0,910 / 1,000	0,984 / 1,000	0,908 / 0,990	0,882 / 0,961
Cải tiến 1: LaBSE + DP	0,882 / 0,986	0,984 / 1,000	0,888 / 1,000	1,000 / 1,000
Cải tiến 2: LaBSE + HV + DP	0,910 / 1,000	0,984 / 1,000	0,888 / 0,980	–

Bảng 4 và 5 cho thấy CroCoAlign chạy zero-shot chỉ đạt F_1 strict trong khoảng 0,53–0,57 trên gold gán tay, thấp hơn rất nhiều so với con số 87,5 mà tác giả báo cáo trên Opus Books; dữ liệu của chúng tôi rõ ràng nằm ngoài phân phối mà mô hình được huấn luyện. Việc tinh chỉnh hai siêu tham số suy luận trên DEV mang lại thêm 3,6 điểm trên TEST, gần như trùng với mức tăng 3,7 điểm quan sát được trên DEV, nên có thể nói tinh chỉnh này không bị overfit. Đáng chú ý hơn cả là mọi hệ dùng quy hoạch động đơn điều đều vượt CroCoAlign từ 31 đến 40 điểm, và khi giữ nguyên bộ mã hoá LaBSE của checkpoint mà chỉ đổi bộ giải mã, F_1 strict đi từ 0,529 lên 0,918.

8 Phân tích

8.1 Vấn đề nằm ở bộ giải mã

Bảng 6 liệt kê vài lỗi tiêu biểu của CroCoAlign nguyên bản trên mục 7, nơi 33 trong 75 câu Hán bị đóng sai. Có thể nhận ra ba kiểu lỗi. Kiểu thứ nhất là bỏ sót những câu dài có nội dung rõ ràng (zh_3, zh_7, zh_8): xác suất mà đầu phân loại đưa ra thấp hơn 0,5 dù cosine LaBSE của cặp đúng khá cao. Kiểu thứ hai là nối nhầm sang một câu ở rất xa nhưng cùng chủ đề, như zh_9 bị nối với vi_56 cách đó 46 câu; điều này chỉ xảy ra được vì mỗi cặp được quyết định độc lập, không có ràng buộc nào buộc câu sau phải nằm sau câu trước. Kiểu thứ ba là chỉ bắt được một phần của nhóm một-nhiều (zh_6 chỉ nối với vi_6). Cả ba kiểu lỗi đều biến mất khi giữ nguyên embedding nhưng giải mã bằng quy hoạch động: ràng buộc đơn điều loại bỏ các ứng viên ở xa, và việc cộng dồn điểm dọc theo đường đi bù đắp cho những cặp có độ tương đồng chỉ ở mức vừa phải. Bản dịch ĐVSKTT bám rất sát thứ tự bản Hán (trong 185 nhóm gold không có nhóm nhiều-nhiều chéo nào), nên giả thiết đơn điều gần như đúng tuyệt đối. Đây là điểm khác căn bản so với những tài liệu văn học nhiều nhiều mà CroCoAlign được thiết kế để xử lý.

Bảng 6: Một số lỗi của CroCoAlign nguyên bản trên mục 7 Sĩ Vương, đối chiếu với gold gán tay

Câu Hán	Gold	CroCoAlign dự đoán
zh_5 土王紀	vi_4 “KỶ SĨ VƯƠNG.”	vi_5 “SĨ VƯƠNG.” (tiêu đề ngắn, nhầm câu kè)
zh_9 父賜漢桓帝時爲日南太守	vi_10 “Cha là Tứ, thời Hán Hoàn Đế làm Thái thú Nhật Nam.”	vi_56 “(Tân nhận chức năm Kiến An thứ 6 thời Hán).”, cách 46 câu
zh_7 姓土諱變字彥威蒼梧廣信人也	vi_8 “Họ Sĩ, tên húy là Nhiếp, tự là Ngạn Uy...”	∅ (bỏ sót)
zh_6 土王在位四十年壽九十年王寬厚...	vi_5 + vi_6 + vi_7 (một-ba)	vi_6 (chỉ “Ở ngôi 40 năm, thọ 90 tuổi.”)

8.2 Tín hiệu tương đồng không còn là yếu tố quyết định

Khi đã dùng chung bộ giải mã, khoảng cách giữa các nguồn tín hiệu thu hẹp đáng kể: baseline độ dài đạt 0,879, từ vựng Hán-Việt 0,934, LaBSE 0,918 và hợp nhất 0,927. Ba hệ đứng đầu chỉ chênh nhau không quá ba nhóm trên tổng số 185, nằm trong sai số của một tập kiểm tra nhỏ. Nói cách khác, trên dữ liệu này bộ mã hoá đa ngữ không tỏ ra hơn tín hiệu từ vựng sẵn có ở nguồn. Phiên bản hợp nhất với $w = 0,3$ (tức là tín hiệu từ vựng chiếm 70%) có nhỉnh hơn LaBSE thuần nhưng vẫn chưa vượt được baseline từ vựng. Một phần nguyên nhân là với văn ngôn thế kỷ XV, cosine LaBSE của những cặp đúng thường chỉ vào khoảng 0,5–0,7 (chẳng hạn 按黃帝時建萬國 và “Xét: Thời Hoàng Đế dựng muôn nước” có cosine 0,52), thấp hơn nhiều so với các cặp ngôn ngữ hiện đại.

8.3 Cách tính điểm gộp

Trong tất cả các fold của tất cả các hệ, cross-validation đều chọn $c = 1$, tức là tính điểm gộp trên văn bản ghép. Với cách lấy trung bình, một nhóm một-hai trong đó có một câu Việt ngắn (“Hữu ty hỏi vì có gì?”) thường bị tách thành một cặp một-một và một câu bỏ trống, vì ô của câu ngắn kéo trung bình xuống dưới ngưỡng τ . Ghép hai câu lại rồi mới so sánh thì độ tương đồng của cả nhóm được bảo toàn. Riêng với baseline Hán-Việt, thay đổi này đưa F_1 strict từ 0,859 lên 0,934 và số nhóm sai từ 24 xuống 11.

8.4 Gold bạc thiên vị về cấu trúc

Có hai bằng chứng cho nhận định này. Thứ nhất, hệ LaBSE+DP đạt đúng 1,000 trên DEV bạc, vì bộ gold đó là điểm bất động của chính nó; hệ quả là khi chọn cấu hình trên DEV, kết quả trên TEST chỉ còn 0,844 so với 0,918 khi chọn bằng cross-validation, tức là chênh 7,4 điểm chỉ do cách chọn. Thứ hai, trên DEV bạc, cách gộp trung bình cho kết quả cao hơn cách gộp văn bản ghép (0,933 so với 0,882), nhưng trên gold gán tay thì ngược lại (0,859 so với 0,934); lý do đơn giản là gold bạc được sinh bởi đúng thuật toán gộp trung bình. Từ đó chúng tôi cho rằng những con số chỉ dựa trên gold bạc, như ở phiên bản đầu của đề tài, không nên được coi là kết quả thật, và gold gán tay là bắt buộc khi hệ được chấm và hệ sinh gold có cùng nguồn gốc.

8.5 Lỗi còn lại và hạn chế

Hệ tốt nhất còn sai 11 trong 185 nhóm, và tất cả đều là nhóm một- n với $n \geq 3$ (các chú giải dài, lời bình của sử thần được tách thành nhiều câu Việt; block về bóng nêu ứng với bảy câu Việt) hoặc nhóm hai-hai (niên đại và lời tâu bị cắt lệch nhau). Những nhóm này nằm ngoài tập bước của bộ giải mã, nên lỗi không đến từ tín hiệu tương đồng. Về hạn chế của nghiên cứu, bộ gold 185 nhóm trên ba mục còn nhỏ (ba nhóm tương đương khoảng 1,6 điểm F_1) và do một người gán, chưa có số đo đồng thuận; cross-validation ba fold trên ba mục cũng còn thô; CroCoAlign tinh chỉnh vẫn được chọn cấu hình trên DEV bạc; và chúng tôi chưa thử fine-tune bộ mã hoá ngữ cảnh của CroCoAlign trên cặp Hán cổ – Việt.

9 Kết luận

Chúng tôi đã hoàn thành ba yêu cầu của đề tài. Về dữ liệu, pipeline cào và tiền xử lý xử lý được 14 mục ĐVSKTT, trong đó bài toán tách câu Hán cổ được giải quyết gọn nhờ dấu câu của phần phiên âm. Về mô hình, CroCoAlign được cài đặt và chạy lại từ mã nguồn gốc với checkpoint chính thức trên stack phần mềm hiện đại, không sửa mã gốc và chạy được trên cả GPU lẫn CPU. Về đánh giá, chúng tôi xây dựng gold gán tay độc lập với mô hình, chọn cấu hình bằng cross-validation, so sánh với baseline và đề xuất cải tiến. Kết quả chính là CroCoAlign zero-shot đạt F_1 strict 0,529 (0,565 sau tinh chỉnh), trong khi giữ nguyên bộ mã hoá của nó nhưng giải mã đơn điệu đạt 0,918, và một baseline thuần từ vựng Hán-Việt đạt 0,934. Hai bài học rút ra là với những cặp văn bản dịch sát, ràng buộc thứ tự quan trọng hơn chất lượng embedding, và gold sinh từ một mô hình không dùng được để đánh giá các mô hình cùng gốc. Trong thời gian tới, chúng tôi dự định mở rộng bộ gold và đo độ đồng thuận giữa người gán, bổ sung các bước một-ba và hai-hai cho bộ giải mã, và thử fine-tune CroCoAlign trên ngữ liệu Hán cổ – Việt sinh ra từ chính pipeline này.

Tài liệu

- [1] F. M. Molfese, A. Bejgu, S. Tedeschi, S. Conia, R. Navigli. CroCoAlign: A Cross-Lingual, Context-Aware and Fully-Neural Sentence Alignment System for Long Texts. *Proc. EACL 2024 (Long)*, 2209–2220. <https://aclanthology.org/2024.eacl-long.135/>. Mã nguồn: <https://github.com/>

Babelscape/CroCoAlign.

- [2] B. Thompson, P. Koehn. Vecalign: Improved Sentence Alignment in Linear Time and Space. *Proc. EMNLP-IJCNLP 2019*, 1342–1348.
- [3] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, W. Wang. Language-agnostic BERT Sentence Embedding. *Proc. ACL 2022*, 878–891.
- [4] W. A. Gale, K. W. Church. A Program for Aligning Sentences in Bilingual Corpora. *Computational Linguistics* 19(1):75–102, 1993.
- [5] R. Sennrich, M. Volk. Iterative, MT-based Sentence Alignment of Parallel Texts. *Proc. NODALIDA 2011*.
- [6] Ngô Sĩ Liên và các sử thần triều Lê. *Đại Việt Sử Ký Toàn Thư*. Bản fulltext Hán – phiên âm – dịch, Vietnamese Nôm Preservation Foundation. <https://www.nomfoundation.org/nom-project/history-of-greater-vietnam/Fulltext?uiLang=vn>

A Hướng dẫn tái lập

Gói nộp (kho CK_XLNNTN, nhánh eval-manual-gold): *dataset* – data/raw (thô theo trang), data/processed (đầu vào CroCoAlign), data/gold_manual + data/annotation (gold gắn tay và phiếu); *source* – scripts/ (toàn bộ do nhóm viết), CroCoAlign/ (submodule mã gốc); *model* – không huấn luyện mô hình mới; dùng checkpoint chính thức của tác giả (Google Drive id 1DwOAB50loUc0lBe6gImX8TI7RqxD8XCw, 2,31 GB) và LaBSE trên HuggingFace (sentence-transformers/LaBSE); các hệ cải tiến là phi tham số (cấu hình trong data/results/*.json), *embedding cache* data/emb/; *kết quả* – data/pred/, data/results/, RESULTS.md.

Môi trường (macOS arm64 hoặc Linux; GPU không bắt buộc):

```
git clone --recurse-submodules <repo> && cd CK_XLNNTN && git checkout eval-manual-gold
uv venv --python 3.10 .venv
uv pip install -p .venv/bin/python torch transformers sentence-transformers faiss-cpu
    jsonlines \
    scikit-learn hydra-core==1.3.2 omegaconf==2.3.0 torchmetrics pytorch-lightning
    GitPython python-dotenv
uv pip install -p .venv/bin/python --no-deps nn-template-core==0.1.1
uvx gdown 1DwOAB50loUc0lBe6gImX8TI7RqxD8XCw -O CroCoAlign/checkpoints/crocoalign.ckpt
```

Phiên bản đã kiểm chứng: Python 3.10, torch 2.14.0, transformers 5.16.1, sentence-transformers 6.0.1, pytorch-lightning 2.6.5, faiss-cpu 1.x, nn-template-core 0.1.1. Suy luận là tắt định (không seed).

Bước 1 – dữ liệu (thuần Python, không cần thư viện ngoài; cào ≈5 phút):

```
python3 scripts/scrape_dvsktt.py --list # liet ke 73 muc
python3 scripts/scrape_dvsktt.py --out data/raw --delay 0.8 --sections 1-Ky-Hong-Bang-thi
    ... 14-Ky-nha-Ngo
python3 scripts/preprocess.py --raw data/raw --out data/processed
```

Bước 2 – gold (đã có sẵn; chạy lại khi sửa file gắn nhãn):

```
python3 scripts/make_annot_sheet.py --section 7-Ky-Si-Vuong --out data/annotation/7-Ky-Si-
    -Vuong.sheet.md
python3 scripts/annot2gold.py --section 7-Ky-Si-Vuong \
    --align data/annotation/7-Ky-Si-Vuong.align.txt --out data/gold_manual/7-Ky-Si-Vuong.
    jsonl
bash scripts/build_silver_all.sh # silver DEV (can LaBSE)
```

Bước 3 – chạy toàn bộ hệ thống và chấm điểm (một lệnh, ≈ 15 phút CPU; trên WSL/GPU đặt PY tới env crocoalign):

```
PY=.venv/bin/python bash scripts/run_wsl_all.sh
```

Lệnh này lần lượt: CroCoAlign gốc và tuned trên TEST (run_eval.py, run_eval_tuned.py $\rightarrow$ data/pred/crocoalign_*_test/results_*.tsv); run_improved.py sinh 64 cấu hình LaBSE(+Hán-Việt)+DP cho 14 mục; grid_baseline.py sinh 12 cấu hình cho mỗi baseline; pick_config.py --cv chọn cấu hình và ghi data/results/<hệ>_test.json; make_results_table.py chèn bảng vào RESULTS.md. Chấm một hệ bất kỳ: python3 scripts/score.py --gold-dir data/gold_manual --pred-dir <thư mục dự đoán>.

Bước 4 – báo cáo: cd report && tectonic bao_cao.tex (XeLaTeX; font Times New Roman và Songti SC của macOS – trên hệ khác thay bằng TeX Gyre Termes và Noto Serif CJK SC).